GURU NANAK INSTITUTIONS TECHNICAL CAMPUS

(An UGC Autonomous Institution - Affiliated to JNTUH)
Ibrahimpatnam, Ranga Reddy (District) - 501 506.

Project Title(MPWB06)

E-Commerce Website Using Django

Domain Name: Full Stack with Python
Batch id:
Guide: Mr.K.Manirathnam Babu Sir

Team Members:
PUDARI DEEPAK (22WJ8A6743)
SAMINENI MIDHUN (22WJ8A6748)
NEHA MUNDADA (22WJ8A6737)
MAROJU VAISHNAVI (22WJ8A6737)

Problem statement:

In the digital age, small and medium-sized businesses often struggle to establish an online presence due to the complexity and cost of building e-commerce platforms. There is a need for a customizable, scalable, and user-friendly web application that allows businesses to showcase products, manage inventory, and process customer orders efficiently.

Solution statement:

This project builds a modular full-stack e-commerce site using Django, featuring user authentication, product browsing, shopping cart, order placement, and an admin dashboard for managing products and orders.. It leverages Django's ORM, templating engine, and admin interface for secure, maintainable, and rapid development, making it ready for real-world deployment and future enhancements. The solution is designed to be modular and extensible, making it suitable for real-world deployment and further enhancement.

ABSTRACT

This project aims to develop a Django-based e-commerce that provides a seamless online shopping experience. The system will include user authentication, product listings, a shopping cart, and an order management system. Users can browse products, add them to the cart, and complete purchases, while administrators can manage products, categories, and orders efficiently.

The website will be built using Django (Python) for the backend, along with HTML, CSS, and JavaScript for the frontend. Django's built-in ORM will handle database interactions, ensuring efficient product and user data management. Payment integration can be added using third-party gateways for secure transactions.

EXISTING SYSTEM

- The proposed system aims to develop a Flask-based portfolio website that is lightweight, flexible, and efficient. It will include key features such as a Projects Section for showcasing past work, a Skills Showcase for highlighting expertise, and a Contact Form for direct communication via email without database dependency.
- Built with Flask, HTML, CSS, and JavaScript for a responsive and user-friendly interface and it Offers easy customization and simplified maintenance.
- Flask's minimalistic design improves performance and reduces complexity compared to Django:Migration results in a streamlined, professional portfolioSupports self-branding, networking, and job opportunities.
- Future enhancements may include: Blog integration,real-time analytics,admin panel.

EXISTING ALGORITHM: Flask

ALGORITHM DEFINATION:

- Routing: Flask Maps URLs to functions via @app.route(),decorator.
- Request Handling – It processes HTTP requests (GET, POST, etc.), retrieves data, and executes the required logic.
- Template Rendering: Uses Jinja2 to generate dynamic HTML
- Response Generation – Sends responses back to the user, which can be HTML pages, JSON data, or API responses.

DRAWBACKS:

- Limited Built-in Features

- More Development Effort

- Scalability challenges.

MINIMUMSYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS

- PROCESSOR : Pentium i3 Processor
- RAM : 2GB DD RAM
- HARD DISK : 250 GB

SOFTWARE REQUIREMENTS

- BACK END : PYTHON
- OPERATING SYSTEM : WINDOWS 7
- IDE : Spider3

PROPOSED SYSTEM

- The existing system is a Django-based portfolio website with features like project showcasing, skill representation, and a contact form. While Django offers a structured framework with built-in database support, it may add unnecessary complexity for a simple portfolio.

- Migrates from Django to Flask, retaining Projects, Skills Showcase, and Contact Form without database dependency

- Built with Flask, HTML, CSS, and JavaScript for a lightweight, responsive, and efficient experience

- Enhances customization and performance, with future plans for blog, analytics, and admin panel for self-branding

PROPOSED ALGORITHM:Django

ALGORITHM DEFINATION:

-Model (M): Defines database schema using Django ORM

-View (V): Processes requests, applies logic, and connects models to templates

-Template (T): Renders dynamic frontend using HTML, CSS, and Django's templating engine

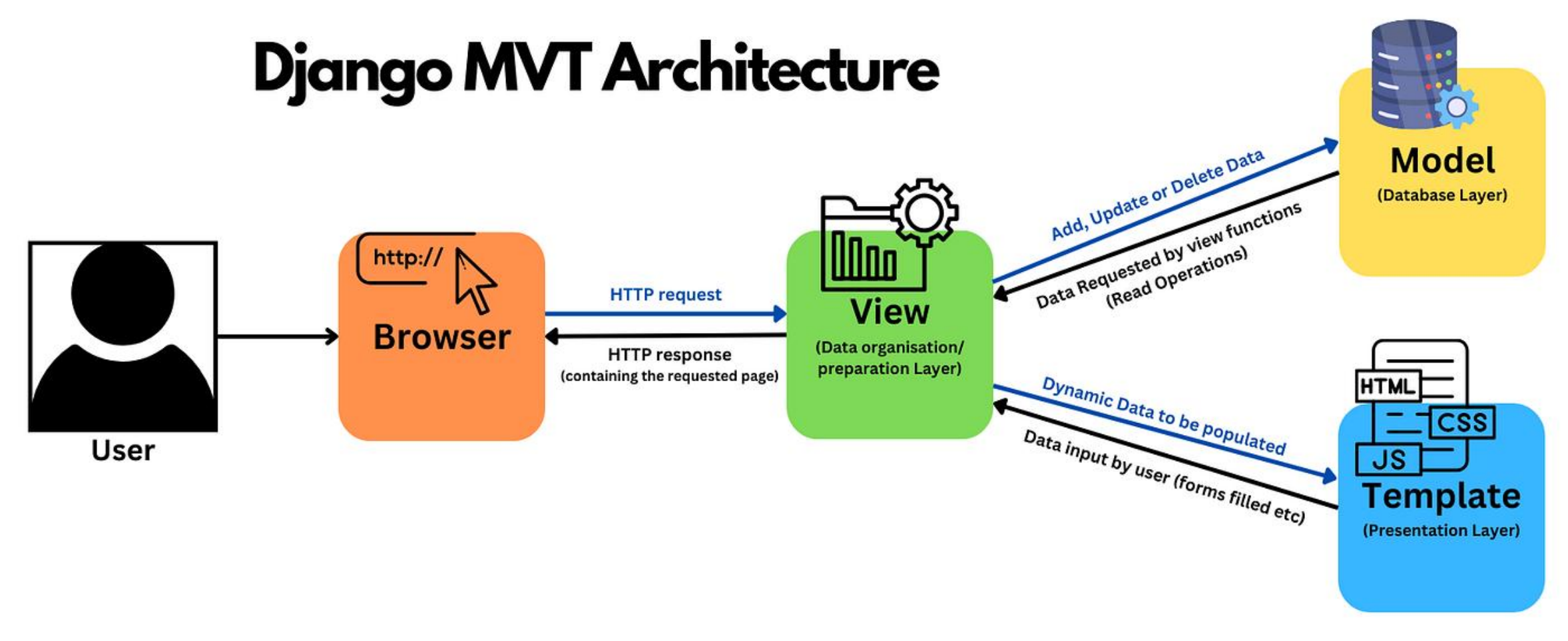
-URL Routing: Directs user requests to views via Django's dispatcher

Advantages:

- Rapid Development

- Security

SYSTEM ARCHITECTURE



Proposed methodogy

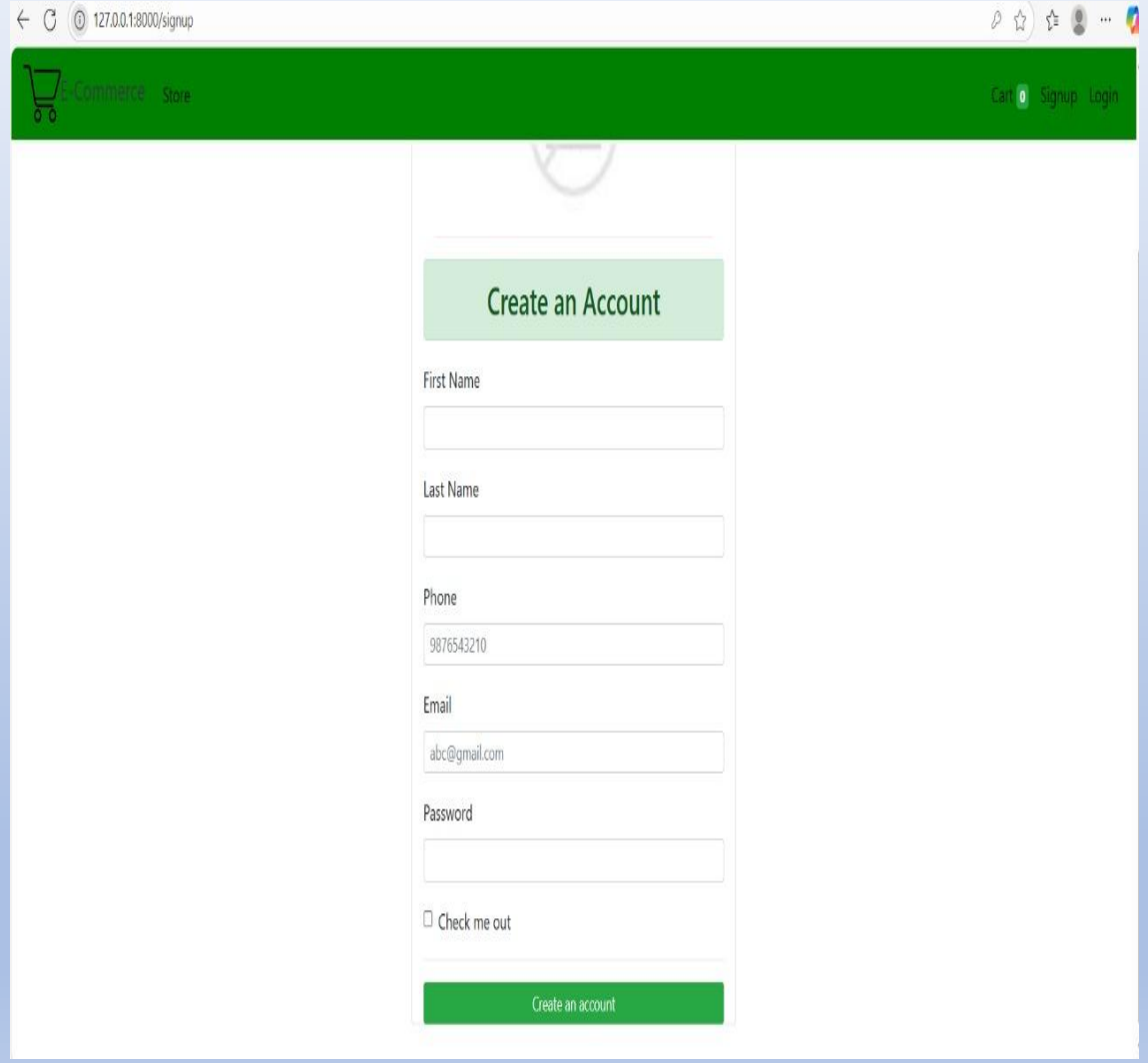
CODE AND IMPLEMENATION

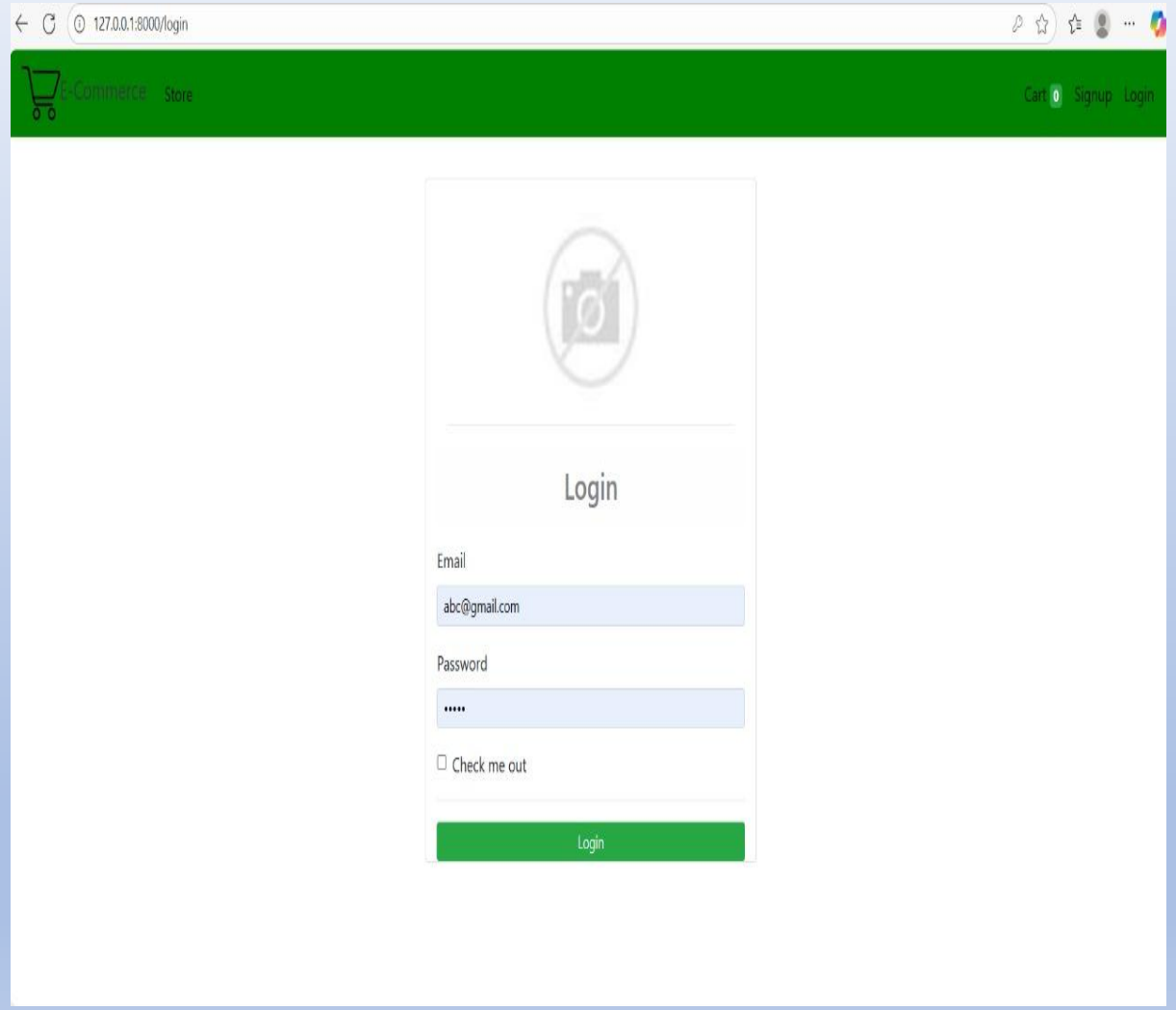
- Implementation:path

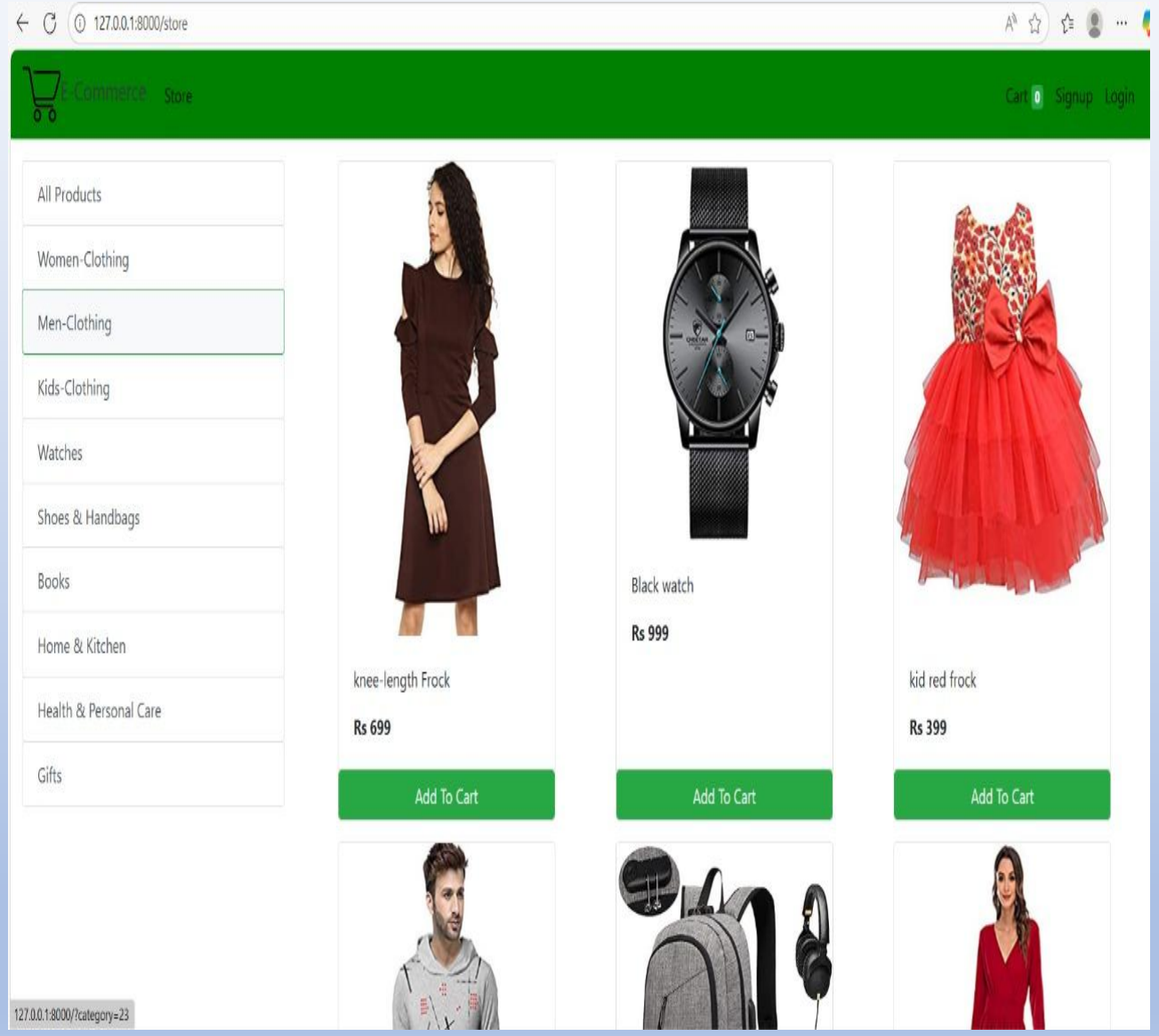
C:\Users\dpudh\Desktop\MPWB06P(2)\MPWB06P(2)\MPWB06P\E-commerceWebsite_using_Django

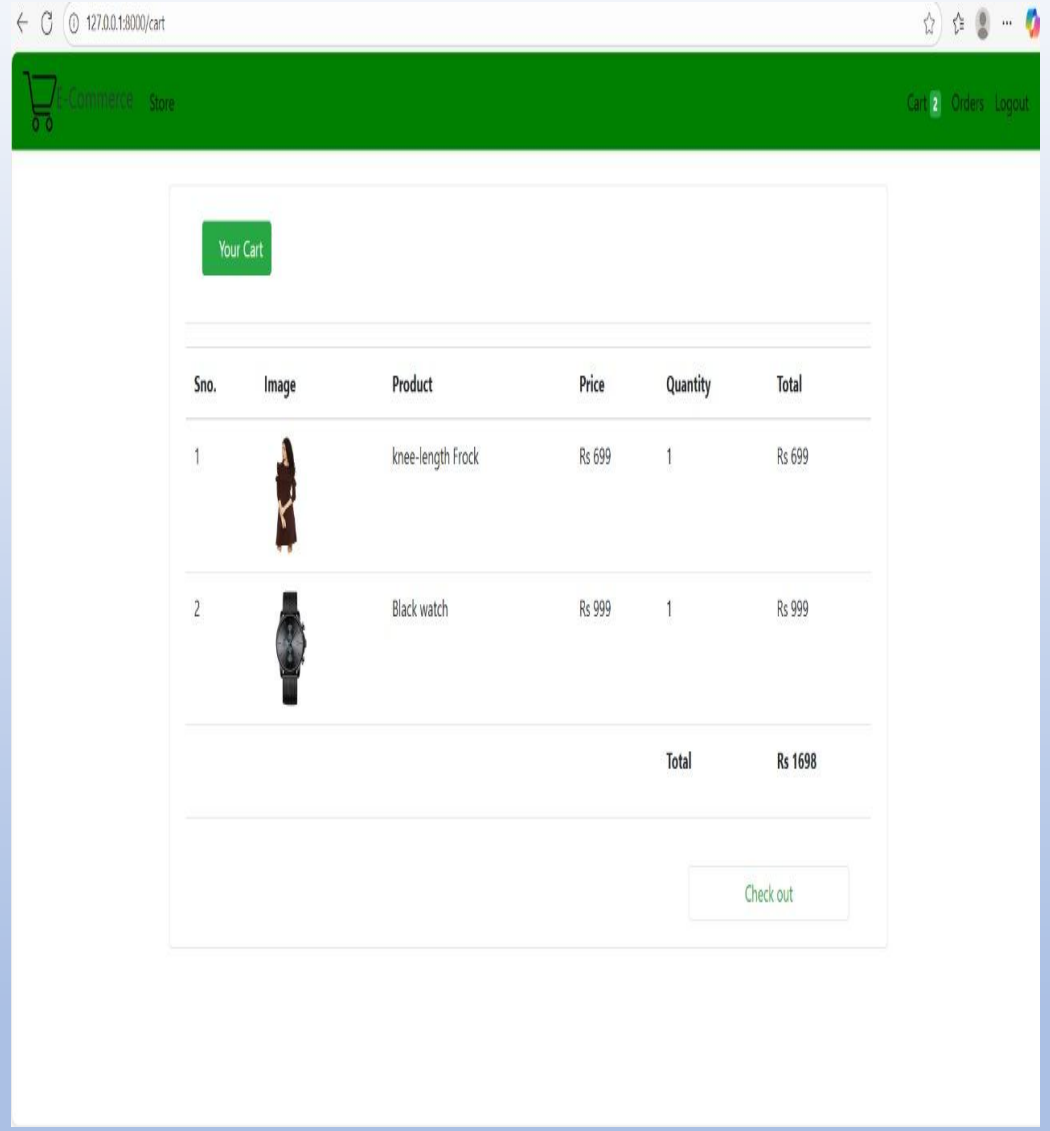
```
Anaconda Prompt - python n

(base) C:\Users\dpudh>cd C:\Users\dpudh\Desktop\MPWB06P(2)\MPWB06P(2)\MPWB06P\E-commerceWebsite_using_Django

(base) C:\Users\dpudh\Desktop\MPWB06P(2)\MPWB06P(2)\MPWB06P\E-commerceWebsite_using_Django>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified some issues:

WARNINGS:
store.Category: (models.W042) Auto-created primary key used when not defining a primary key type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the StoreConfig.default_auto_field attribute to point to a subclass of AutoField, e.g. 'django.db.
models.BigAutoField'.
store.Customer: (models.W042) Auto-created primary key used when not defining a primary key type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the StoreConfig.default_auto_field attribute to point to a subclass of AutoField, e.g. 'django.db.
models.BigAutoField'.
store.Order: (models.W042) Auto-created primary key used when not defining a primary key type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the StoreConfig.default_auto_field attribute to point to a subclass of AutoField, e.g. 'django.db.
models.BigAutoField'.
store.Products: (models.W042) Auto-created primary key used when not defining a primary key type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the StoreConfig.default_auto_field attribute to point to a subclass of AutoField, e.g. 'django.db.
models.BigAutoField'.

System check identified 4 issues (0 silenced).
October 14, 2025 - 22:12:15
Django version 5.2.5, using settings 'Eshop.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[14/Oct/2025 22:12:25] "GET / HTTP/1.1" 302 0
you are :  None
[14/Oct/2025 22:12:25] "GET /store HTTP/1.1" 200 43234
Not Found: /favicon.ico
[14/Oct/2025 22:12:26] "GET /favicon.ico HTTP/1.1" 404 4410
None
/cart
[14/Oct/2025 22:12:50] "GET /cart HTTP/1.1" 302 0
[14/Oct/2025 22:12:50] "GET /login?return_url=/cart HTTP/1.1" 200 5146
[14/Oct/2025 22:12:54] "GET /signup HTTP/1.1" 200 6180
```

SAMPLE OUTPUT






127.0.0.1:8000/store
E-Commerce
Store
Cart 0
Signup
Login
All Products
Women-Clothing
Men-Clothing
Kids-Clothing
Watches
Shoes & Handbags
Books
Home & Kitchen
Health & Personal Care
Gifts
knee-length Frock
Rs 699
Add To Cart
Black watch
Rs 999
Add To Cart
kid red frock
Rs 399
Add To Cart
127.0.0.1:8000/?category=23


127.0.0.1:8000/cart
E-Commerce
Store
Cart 2
Orders
Logout
Your Cart
Sno.
Image
Product
Price
Quantity
Total
1
knee-length Frock
Rs 699
1
Rs 699
2
Black watch
Rs 999
1
Rs 999
Total
Rs 1698
Check out

HOME PAGE

FUTURE SCOPE

- **Scalable Growth**: Django supports large product catalogs and high user traffic.
- **Secure Transactions**: Built-in security features protect user and payment data.
- **Mobile & API Integration**: Easily connects with mobile apps and third-party services.
- **Smart Features**: Can add AI tools like recommendations and chatbots.
- **Analytics & Admin Tools:** Supports dashboards for tracking sales and managing content.

APPLICATIONS

- **Product Listing** – Display products with images, descriptions, and prices
- **Shopping Cart –** Add, update, and remove items before checkout
- **User Login/Register** – Secure authentication and profile management
- **Order Placement** – Process orders and store order history
- **Admin Panel** – Manage products, users, and orders easily
- **Contact Form** – Allow users to send inquiries or feedback

THANKYOU!